

FLEXRANK: Nested Low-Rank Knowledge Decomposition for Adaptive Model Deployment



Why FLEXRANK?

A principled elastic training algorithm for adaptive model deployment across many latency, memory, and quality regimes.

Challenges

Fixed-Cost Monoliths

Pretrained models expose one point on the performance-cost curve; serving a smaller budget means choosing another model.

Deployment Budgets Vary

Devices, request difficulty, memory limits, and latency targets change; not every setting can afford the same active model.

Costly Retraining of Smaller Models

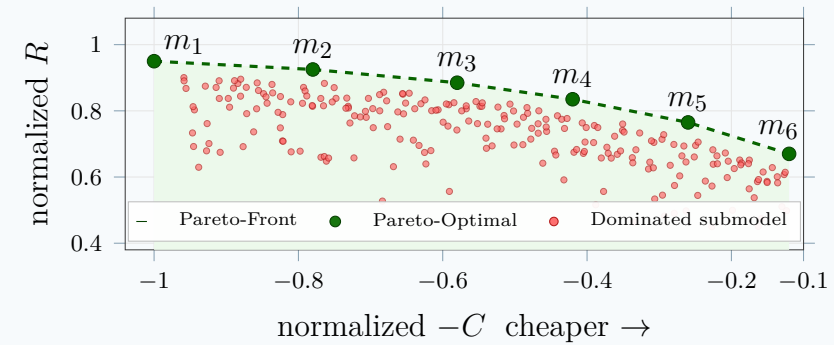
Engineering multiple independent smaller models requires repeated and costly pretraining (e.g. Llama3.2-1B/3B/8B).

Definition: Pareto Elastic Model

Let R_β and C_β be the **reward** and **cost** for the submodel selected by a budget operator $T_\beta(\theta)$. A shared model (f, θ^*) is **Pareto-Elastic** when, for every targeted budget, the selected submodel is non-dominated:

$$\forall \beta \in \mathcal{B}, \quad \nexists (\tilde{\theta}, \tilde{T}_\beta) : \tilde{C}_\beta \leq C_\beta^*, \tilde{R}_\beta > R_\beta^*.$$

The ideal is one parameter vector θ^* plus budget operators T_β^* realizing Pareto Optimal submodels $\{m_k\}_{k=1}^K$ in $(R, -C)$ space.



FLEXRANK approximates the Pareto Front by learning **nested rank prefixes**: higher budgets add residual components instead of swapping to a different model family:

$$m_1 \preceq m_2 \preceq \dots \preceq m_K, \quad T_{\beta_k}(\theta) = \text{rank prefix } m_k.$$

“Train Once, Deploy Everywhere”

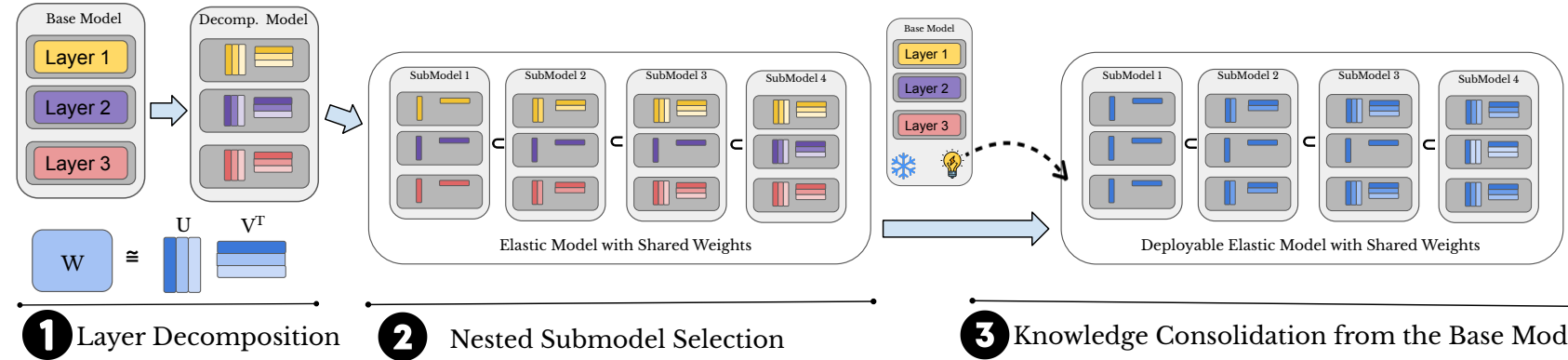
A single decomposed model yields nested submodels for heterogeneous latency, memory, and quality targets.

One Elastic Model
Nested rank prefixes share one set of factors.

Near Pareto-Optimal Submodels
A dynamic programming algorithm selects how to spend parameter budget.

Real Deployment Gains
GAR makes any rank reduction cheaper than full model.

FLEXRANK in One Picture



1. DataSVD Initialization

Calibration-aware SVD ranks directions by data importance.

2. Dynamic Programming Search

Layer probes allocate rank non-uniformly under nested budgets.

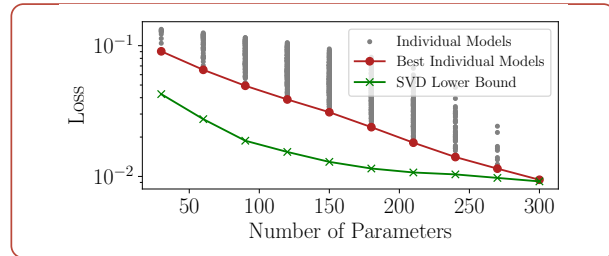
3. Nested Distillation

Selected prefixes learn jointly from one dense teacher.

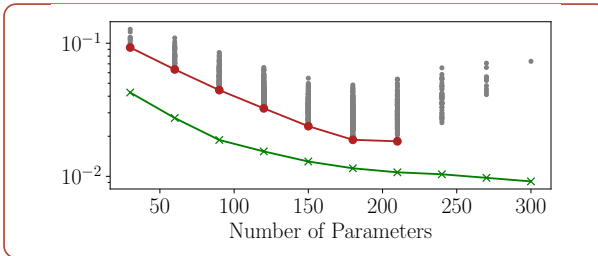
4. Gauge-Aligned Reparametrization (GAR):

$$UV^\top = \underbrace{(UG)}_{\tilde{U}} \underbrace{(G^{-1}V^\top)}_{\tilde{V}^\top}, \quad \tilde{U} = \begin{bmatrix} I_r & \hat{U} \end{bmatrix}^\top, \quad G = (U_{1:r,:})^{-1}, \quad \text{GAR} = (\hat{U}, \tilde{V}^\top), \quad \text{cost}(\text{GAR}) = \mathcal{O}((m+n-r)r) < \mathcal{O}(mn), \quad \forall r < \min(m, n)$$

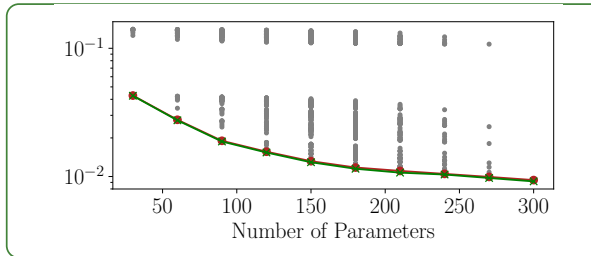
The Need for Nestedness



Post-Training Selection Thm. 4.1
Full-model minimizers almost never contain optimal lower-rank slices.



All-Subspaces Learning Thm. 4.2
All-subspace training gives a positive lower bound on the submodel gap.



Nested Subspace Learning Thm. 4.3
Nested prefixes make each larger model learn only the residual.



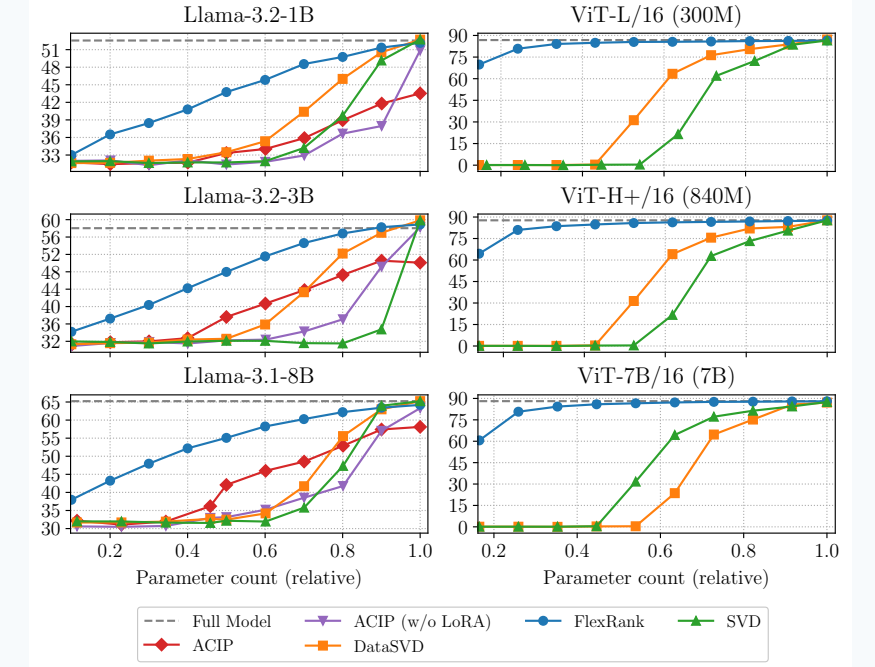
Riccardo Zaccone^{1,2} Stefanos Laskaridis³
Marco Ciccone⁴ Samuel Horváth²

¹Polytechnic of Turin ²MBZUAI
³Amazon Science, UK ⁴Vector Institute



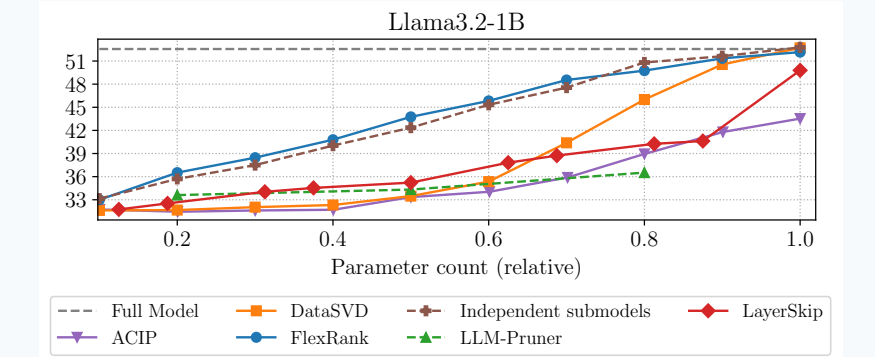
Experimental Results

1. Graceful Degradation Across Budgets



Smooth degradation on commonsense LLM evaluation and ImageNet1K; ViTs stay within 5% of full-model accuracy even after pruning up to 70%.

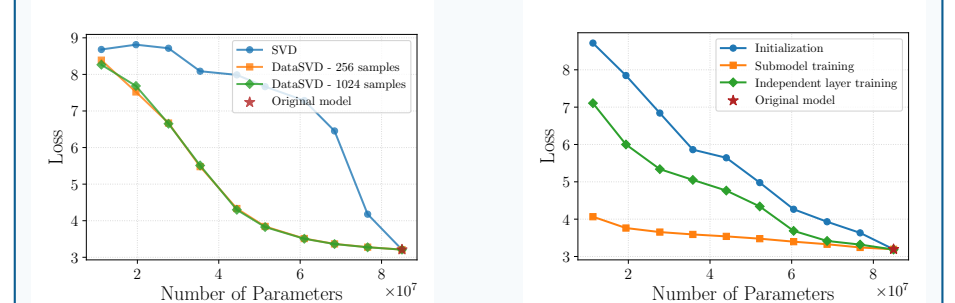
2. Matched-Budget Comparison



Outperforms LLM-Pruner and LayerSkip, and is competitive with independently trained submodels at matched budget; dashed lines denote non-elastic methods.

Ablations

3. Effect of Initialization and Consolidation



DataSVD needs only a few hundred calibration samples; end-to-end nested training consolidates local nestedness into global nestedness.